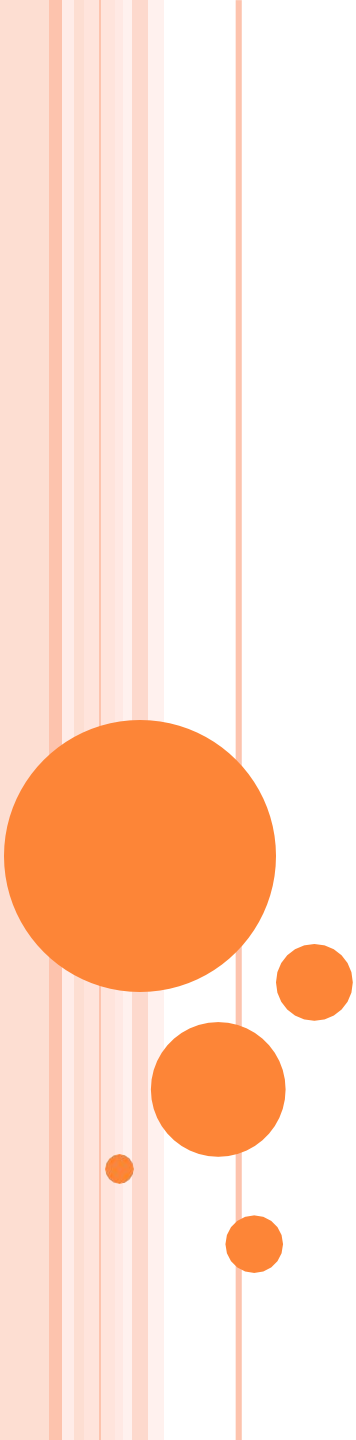




UNIT -1

INTRODUCTION TO JAVA

TOPICS TO BE COVERED

- 1.1 Basics of Java, Background/History of Java, Java and the Internet, Advantages of Java
2. Java Virtual Machine & Byte Code
3. Java Environment Setup
4. Java Program Structure
- 1.5 Procedure-Oriented Vs. Object oriented programming concept
- 1.6 Basics of OOP: Abstraction, Inheritance, Encapsulation, Classes, subclasses and classes, Polymorphism, message communication
- 1.7 Compiling and running a simple "Hello World" program: Setting Up Your Computer, Writing a Program, Compiling, Interpreting and Running the program, Common Errors



1.1 JAVA EVOLUTION

- ❑ Java is a general purpose, class based, and robust, secure, safe, object oriented programming language.
- ❑ It was developed by sun micro system in 1991 by James Gosling its name was **Oak**. In 1995 its name has been changed to **Java** because of some legal issues.
- ❑ Java was designed for development of software for consumer electronic devices like TVs, VCRs, toaster etc.
- ❑ Java removes limitation like portability and reliability of C and C++.
- ❑ Java program would run in fundamentally different execution environments.



- Java is strongly typed language.
- Compile time contains translating programs into a machine independent byte code representation.
- Runtime activities include loading and linking of classes needed to execute a program, optional machine code generation, dynamic optimization of program.
- Most striking feature of java is platform independence. Java is not tied to any particular hardware or OS. Java programs can be executed anywhere on any system.



JAVA FEATURES

There are various features described by sun micro system.

Some of them are given here:

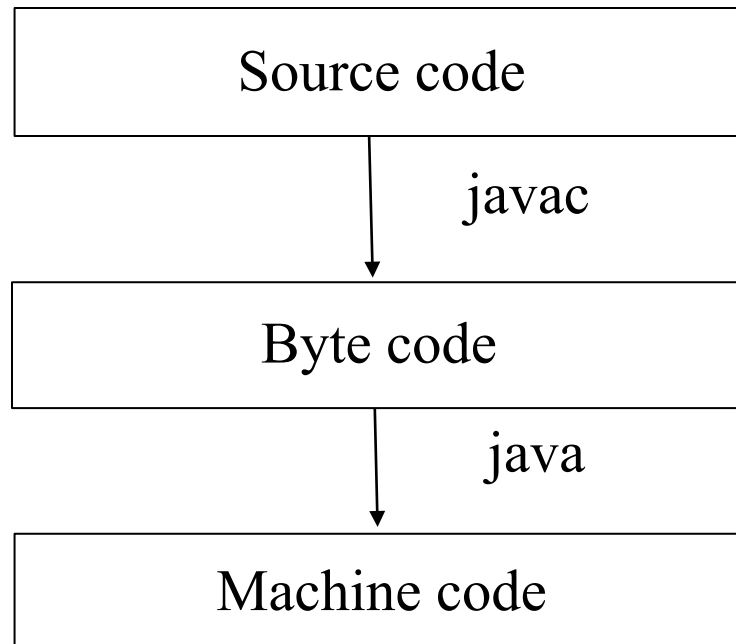
1. **Compiled and interpreted**
2. **Platform independent**
3. **Portable**
4. **Object oriented**
5. **Robust and secure**
6. **Distributed**
7. **Familiar , simple and small**
8. **Multithreaded and interactive**
9. **High performance**
10. **Dynamic and extensible**



1. Compiled and interpreted:

- ❑ Computer language is either compiled or interpreted. Java combines both so it is two stage system
- ❑ First **java compiler** translates **source code** into **byte code**.
- ❑ **Java interpreter** generates machine code that can be

directly executing by machine that is running the program.



2. Platform independent:

- ❑ Programs written by user are called **source code** and executable program is called **object code**.
- ❑ Object can be executed by computer and object is specific to a particular CPU. So it cannot be executed on different platform. Java removes this limitation.
- ❑ When java program is compiled java compiler produces an object file which contains bytecode. These byte codes are not machine/CPU specific, it can be interpreted by JVM (Java Virtual Machine). So that code can be run on any JVM of computer system.
- ❑ The same byte codes can be executed by any JVM on any platform.
- ❑ Thus java is platform independent.



3. Portable :

- Java's important feature is portability.
- Java program can be easily moved from one computer system to another, anywhere and anytime. Changes /upgrade in OS .processors and system resources will not force any changes in java program.
- Java provides Write Once Run Anywhere (WORA), which makes the java language portable provided that the system must have interpreter for the JVM .
- Java also has the standard data size irrespective of operating system or the processor .These features make the java as a portable language.



- Java provides portability in two ways:
- First, java compiler generates byte code instructions that can be implemented on any machine.
- Secondly, the size of primitive data types is machine- independent.



4. Object oriented

- Almost everything in java is an object.
- All program code and data reside within object and classes. Object model in java is simple and easy to extend.
- Java comes with extensive set of classes, arranged in packages that we can use in our program by inheritance.
- Java is true (pure) object oriented language because object is the outer most level of data structure in java.
- Everything in java is object even the primitive data types can also be converted into object by using the wrapper class.



5. Robust and secure

- It provides safe guards to ensure reliable code.
- Java has strict compile time and runtime checking for data types.
- Java also incorporates concept of exception handling, which captures series errors and eliminate any risk crashing system. So, java is robust.
- Absence of pointer in java ensures that program cannot gain access to memory location without proper authorization.



6. Distributed

- ❑ Java is designed as distributed language for creating applications on networks.
- ❑ Java applications can open and access remote objects on internet .These multiple programmers at multiple remote locations to collaborate and work together on a single project.



7. Simple ,small and familiar

- ❑ Java is small and simple.
- ❑ Many features of C and C++ those are redundant or sources of unreliable code are not part of java.
- ❑ It doesn't support pointer,pre-processor , header files,goto statement and other
- ❑ It eliminates operator overloads, multiple inheritance, thus java is simplified version of C++.
- ❑ It includes automatic storage management, typically using a garbage collector, to avoid safety problems of explicit de-allocation of memory references.



8. Multithreaded and interactive

- ❑ Java is multithreaded programming language.
- ❑ Multithreading means a single program having different threads executing independently at the same time. Multiple threads execute instructions according to the program code in process or a program.
- ❑ Java supports multithreaded programs. This means we need not wait for the application to finish one task before beginning other.
- ❑ **Java runtime** comes with tools that support microprocessor synchronization and construct smoothly running interactive system.



9. High performance

- High performance ,by intermediate byte code
- Multithreading enhances the overall execution speed of java program

10. Dynamic and extensible

- Java is dynamic language
- It is capable of dynamically linking in new class libraries, method and object.
- Java program support function written in other language such as C and C++.This function known as native methods.
- Native methods are linked dynamically at runtime.



Note :

Java does not run on Windows 95 and MS-Dos because on DOS and Windows 95 file names have extension having 3(.htm or .txt) character while java file has extension having 4 characters (.java).



1.1 ADVANTAGES OF JAVA

- ❑ Java syntaxes are inherited from C/C++.In C++ many concepts are confused which are left out or more cleared in java.
- ❑ Java is easy to program because it eliminates the drawback of other languages likes pointers, memory management etc.that affect code of robustness.
- ❑ Java is object oriented language, which helps the programmers to visualize the program in real life terms
- ❑ Java enables users to create smooth and clear code.
- ❑ Java is called portable language because it is single code can be run on any operating system.



- ❑ Java is platform independent, so it allows the same software to execute without change on heterogeneous set of devices.
- ❑ Java provides multi-threading, so user can run more than one thread activity at a time.
- ❑ Java codes are checked before loading and so they are secured from run time errors.
- ❑ Java loads classes dynamically, at the time when they are actually needed.
- ❑ Through inheritance, we can eliminate redundant code and extend the use of existing classes.



- ❑ The principle of data hiding helps the programmer to build secure programs that cannot be attacked by code in other parts of program.
- ❑ It is possible to map object in problem domain in the program.
- ❑ The data centred approach enables us to capture more details of a model in an implementable form.
- ❑ Object oriented systems can be easily upgraded from small to large systems.
- ❑ Java strongly associated with Internet.



1.2 JAVA VIRTUAL MACHINE (JVM)

- ❑ Java Virtual Machine or JVM as its name suggest it is a “virtual” machine that resides in the “real” computer as a software.
- ❑ JVM gives Java the flexibility of platform independence. Let us see first how exactly Java program is created, compiled and executed.
- ❑ Java code is written in .java file.
- ❑ This code contains one or more Java language attributes like Classes, Methods, Variable, Objects etc.
- ❑ Javac is used to compile this code and to generate .class file. Class file is also known as “byte code”.
- ❑ The name byte code is given may be because of the structure of the instruction set of Java program.
- ❑ Java byte code is an input to Java Virtual Machine. JVM read this code and interpret it and executes the program



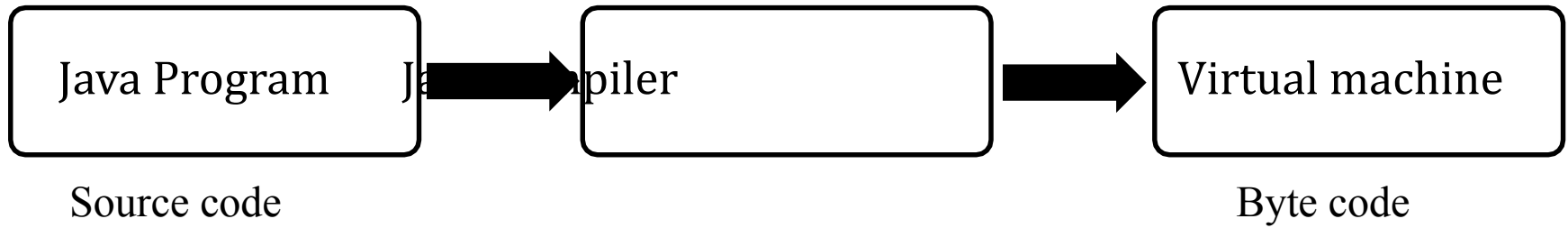


Figure :4.1 Process of compilation

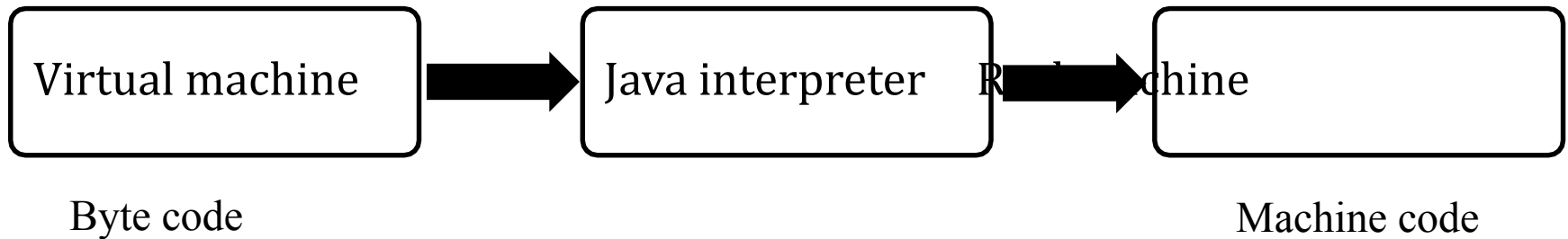
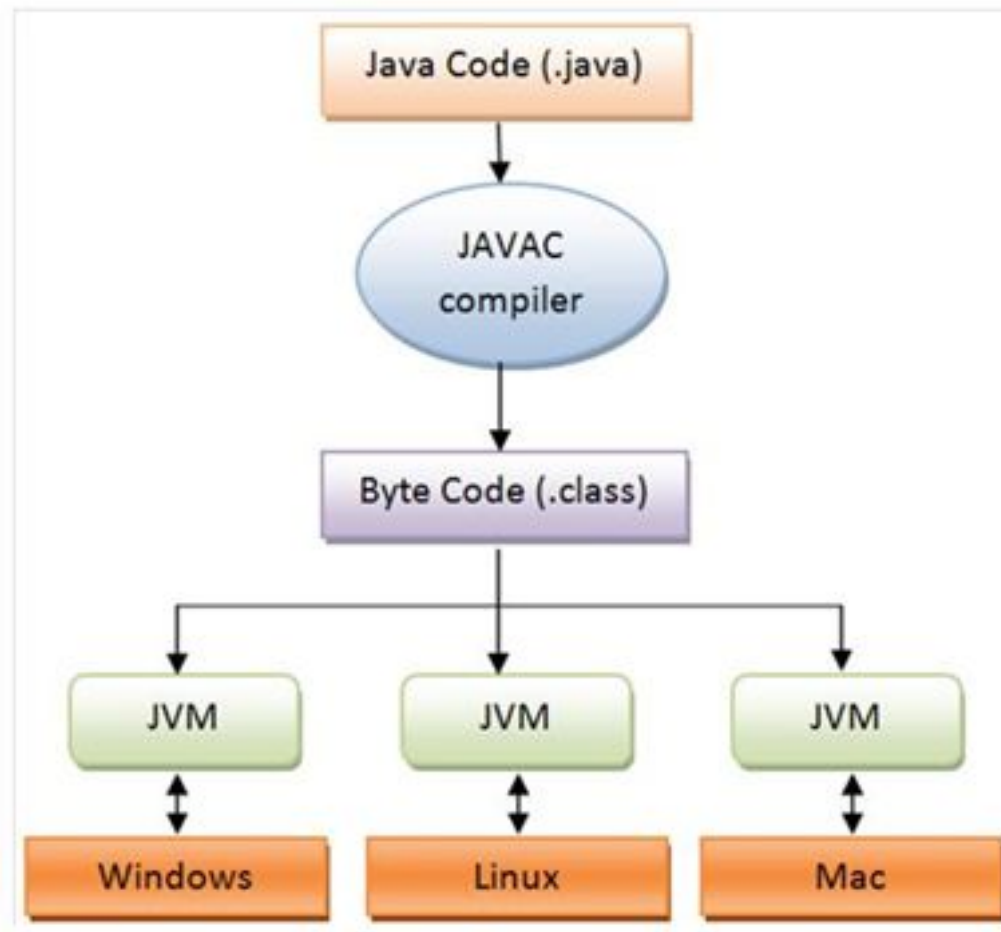


Figure :4.2 Process of converting byte code into machine code



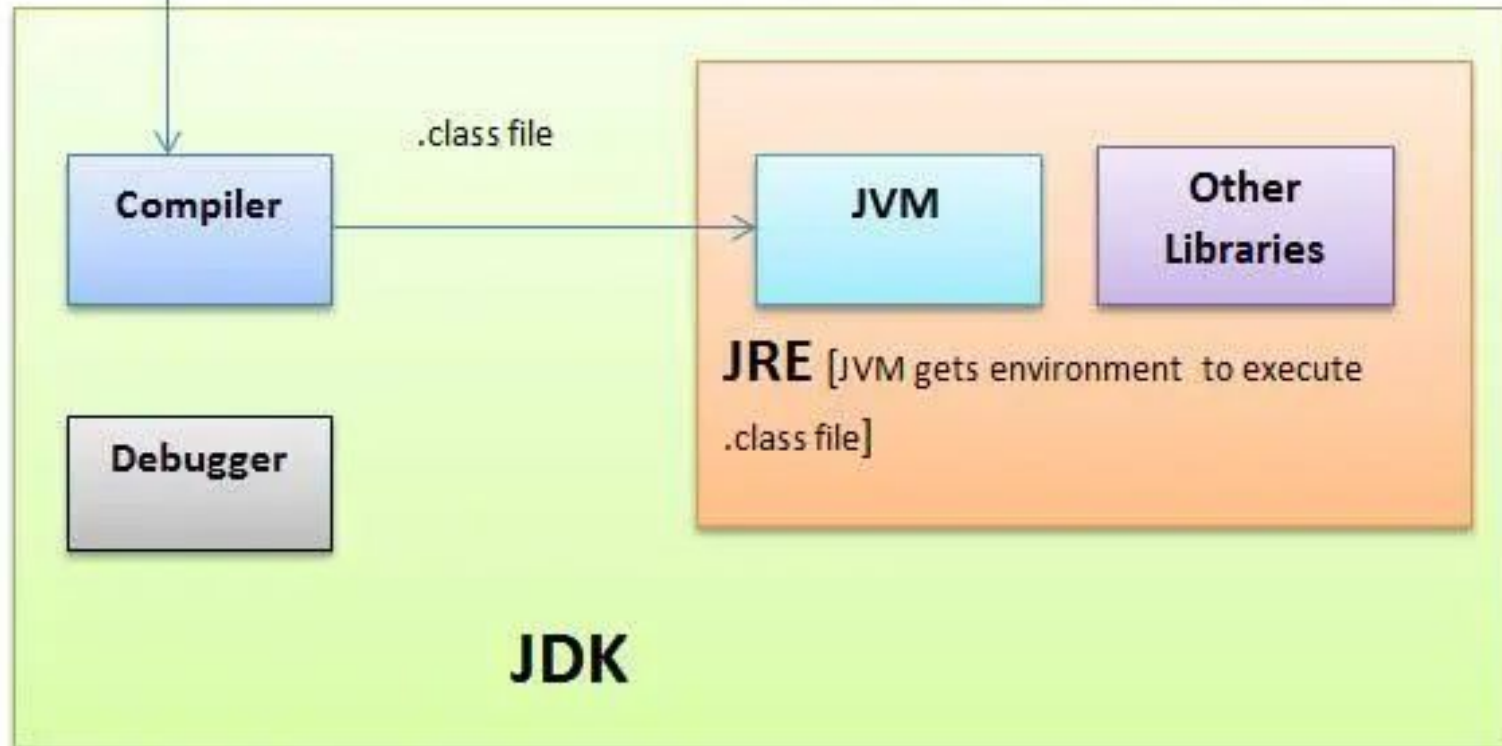


JDK: java development kit

JRE: java Runtime Enviornment

JVM: java Virtual machine

Source files (.java files)



- Java Virtual Machine like its real counterpart, executes the program and generate output.
- To execute any code, JVM utilizes different components.
- JVM is divided into several components like the stack, the garbage-collected heap, the registers and the method area. Let us see diagram representation of JVM.

Figure :5

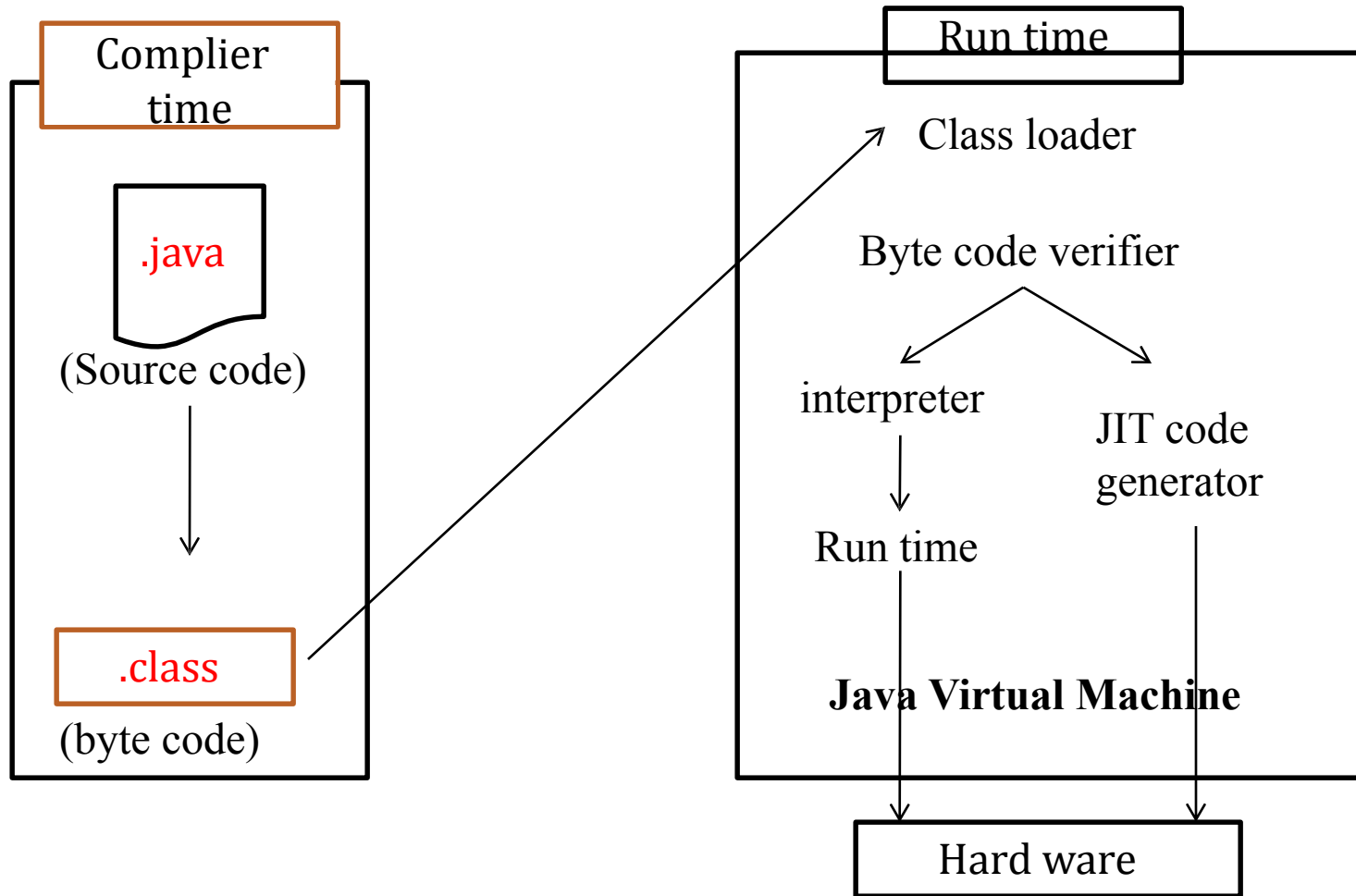
Layer of interactions for java program

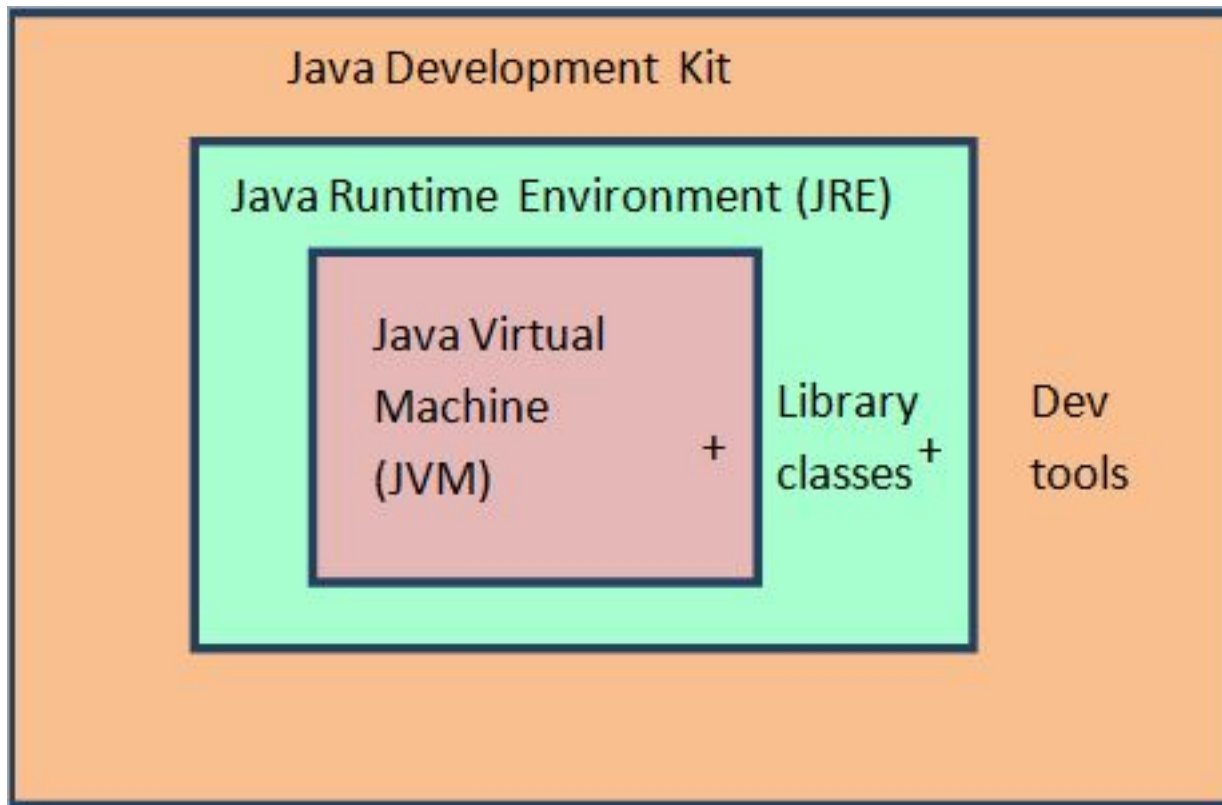


JRE (JAVA RUNTIME ENVIRONMENT)

- JRE consists of JVM and the program on the other .
- It runs code compiled for JVM by:
 - Loading the .class files
 - Verifying byte code
 - Executing the code







- A Java virtual machine is a running software system responsible for executing live Java programs.
- The JRE is the on-disk software component that takes your compiled Java code (the code is compiled with the JDK), combines it with the required libraries, and starts the JVM to execute it.
- The JRE contains libraries and software that your Java programs need to run. As an example, the Java class loader is part of the JRE.
- This important piece of software loads compiled Java code into memory and connects the code to the appropriate Java class libraries (a process known as linking).



1.3 JAVA ENVIRONMENT SETUP JE (JAVA ENVIRONMENT)

- JE includes large no. of development tools and hundred of classes and methods.
- Tools are part of JDK (java development toolkit) and classes and methods are part of JSL (API)
- JE = JSL + JDK

JDK : **J**ava **D**evelopment **T**oolkit JSL

: **J**ava **S**tandard **L**ibrary

API : **A**pplication **P**rogramming **I**nterface

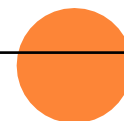


JDK (JAVA DEVELOPMENT TOOLKIT)

- It is a collection of tools that are used for developing and running java program.
- JDK includes
 - **applet viewer** (for viewing java applets)
 - **javac** (java compiler)
 - **java** (java interpreter)
 - **javap** (java disassembler)
 - **javah** (for C header files)
 - **javadoc** (for creating HTML documents)
 - **jdb** (java debugger)



No	Tools	Description
1	appletviewer	It enables user to run java applets.
2	javac (java compiler)	javac translates java source code to byte code files that the interpreter can understand.
3	java (java interpreter)	It runs application by reading and interpreting byte codes.
4	javap(java disassembler)	It enables users to convert byte code files into a program description.
5	javah(for C header files)	It Produces header files for use with native methods.
6	javadoc (for creating HTML Documents)	It creates HTML format documentation from java source code files.
7	jdb (java debugger)	It helps user to find error in our program.



JSL : JAVA STANDARD LIBRARY

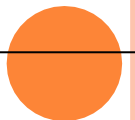
OR

API : APPLICATION PROGRAMMING

INTERFACE

- It includes hundreds of classes and methods grouped into several functional packages.
- Most commonly used packages are:
 - **lang** (language support package)
 - **util** (utility package)
 - **io** (Input/output package)
 - **net** (Networking package)
 - **AWT** (Abstract Window Toolkit package)
 - **applet** (applet package)





No	Package name	Description
1	Language support package (lang)	It includes classes and methods required for <u>implementing basic feature of Java.</u>
2	Utility package (util)	Collection of classes to provide <u>utility functions such as date and time functions.</u>
3	Input output package(io)	A collection of classes required for <u>input/output manipulation.</u>
4	Networking package(net)	Collection of classes for <u>communicating with other computers via internet.</u>
5	Abstract Window toolkit package(awt)	Collection of classes that implements Platform <u>independent Graphical User Interface</u>
6	Applet package(applet)	Collection of classes that <u>allows us to create java applets.</u>

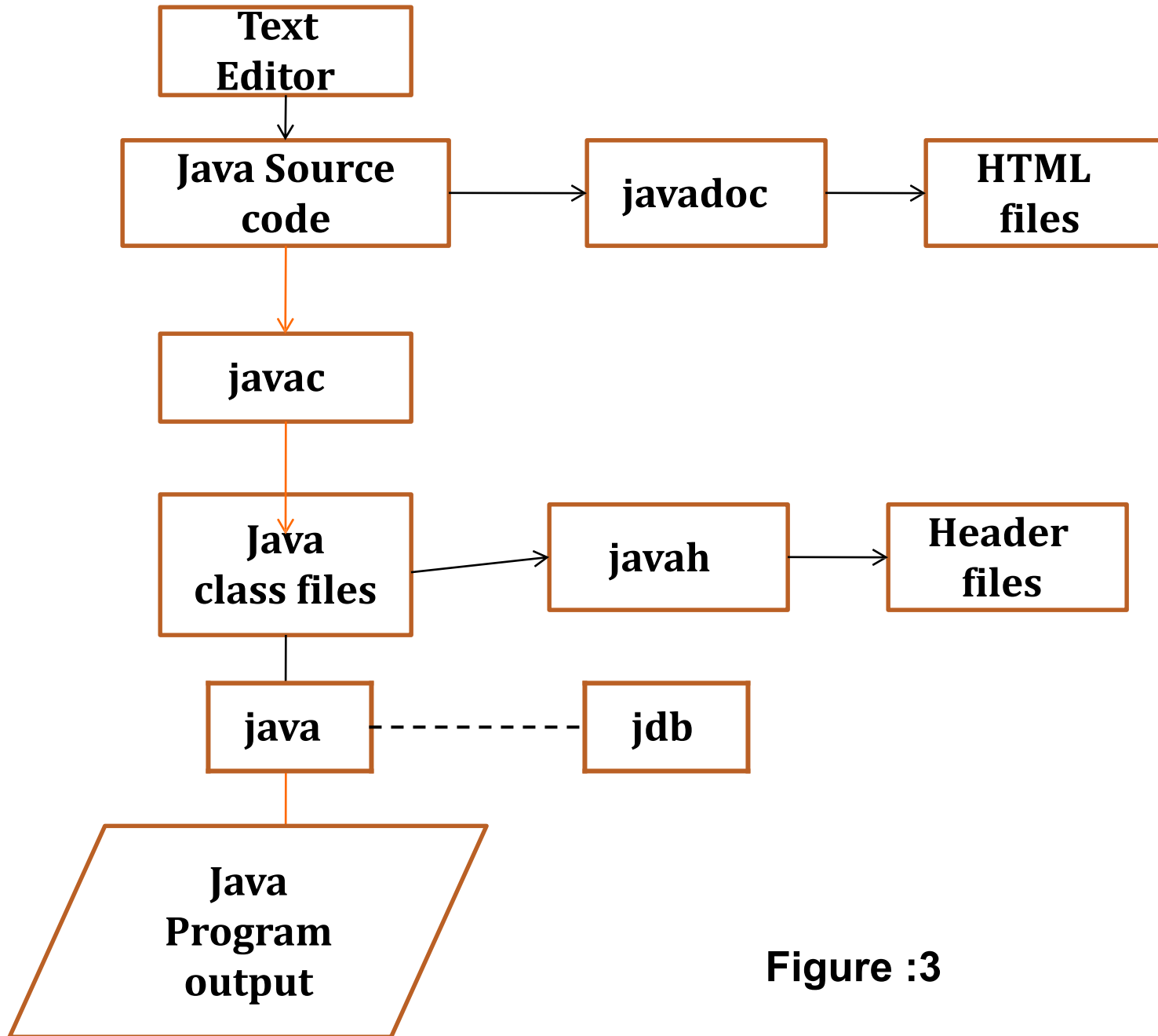


Figure :3



IMPLEMENTING JAVA PROGRAM

- Java application program implementation involves a three steps:
 - Creating the program (`.java` file)
 - Compiling the program(using `javac`)
 - Running the program (using `java`)

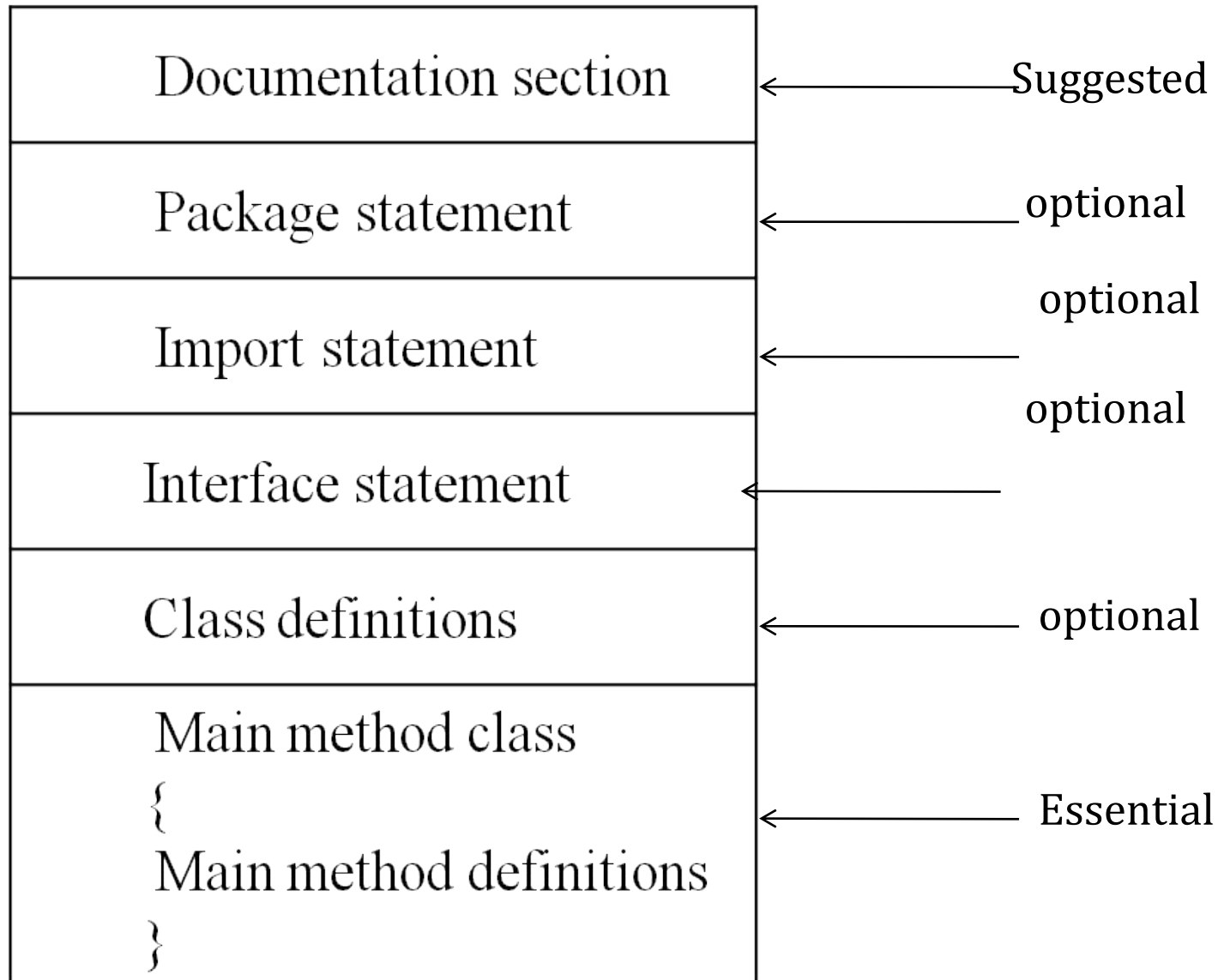


1.4 JAVA PROGRAM STRUCTURE BASIC

STRUCTURE OF JAVA PROGRAM

- ❑ Java program may contain many classes of which only one class defines a main method.
- ❑ Class contain data members and methods that operate on data members of class.
- ❑ Method may contain data type declaration and executable statements.java program define classes and put them together.





Basic structure of java program

DOCUMENTATION SECTION:

- This section comprises a set of comment lines giving the name of the program, author and other details, which the programme would like to refer to at a large stage.
- Comments must explain why and what of classes and how to algorithms.
- Java supports three types of comment line:

● **Single comment line**

: //

● **Multiline** :

/* */

● **Documentation line**

: /** */
/



- Single-line Comment:** It starts with a pair of forwarding slash (//).
For example:
`//First Java Program`
- Multi-line Comment:** It starts with a /* and ends with */. We write between these two symbols. For example:
`/*It is an example of
multiline comment*/`
- Documentation Comment:** It starts with the delimiter (/**) and ends with */. For example:
`/**It is an example of documentation comment*/`



PACKAGE STATEMENT:

- The statement declares a package name and informs the compiler that the classes defined here belong to this package.
- `package student ;   //student is package`

package statement is optional.



IMPORT STATEMENTS:

- After package statement (but before class definitions) be a number of import statements. This is similar to the #include statement in C.

import package.name.classname;

- This statement instructs the interpreter to load the **test** class contained in the package **Student**.
- Using import statement ,we can access to classes that are part of the other named packages.
- `import java.util.Scanner;`



INTERFACE STATEMENT:

- ❑ Interface is like a class but includes a group of method declarations. This is also an optional section.
- ❑ It is used only when we want to implement the multiple inheritance feature in the program.

```
interface car
{
void start();
void stop();
}
```



CLASS DEFINITIONS:

- A java program may contain multiple class definitions. Classes are the primary and essential elements of a Java program. The number of classes used depends on the complexity of the problem.

```
class Student //class definition
{
String sname; //variable
int id;
double percentage;
}
```



MAIN METHOD CLASS:

- Every java stand alone program requires a main method as its starting point, this class is essential part of a java program.
- Simple java program may contain only this part.
- The main method creates objects of various classes and creates communications between them.
- On reaching the end of main ,the terminates and control passes program back to the operating system.



```
public class Student //class definition
{
  public static void main(String args[])
  {

  }

}
```

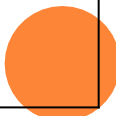


1.5 PROCEDURE-ORIENTED Vs. OBJECT ORIENTED PROGRAMMING CONCEPT



PROCEDURE-ORIENTED Vs. OBJECT ORIENTED PROGRAMMING

No	Procedure oriented programming	Object oriented programming
1	Program is divided into small parts called functions.	Program is divided into parts called objects.
2	POP follows Top Down approach.	OOP follows Bottom Up approach.
3	POP does not have any access specifier.	OOP has access specifier named Public, Private, Protected
4	Adding new data and functions is not easy.	Adding new data and function is easy.
5	Procedural programming does not have any proper way of hiding data so it is less secure.	Object-oriented programming provides data hiding so it is more secure.



6	Procedural programming is used for designing medium-sized programs.	Object-oriented programming is used for designing large and complex programs.
7	In procedural programming, there is no concept of data hiding and inheritance.	In object-oriented programming, the concept of data hiding and inheritance is used.
8	In POP, Overloading is not possible.	Overloading is possible in the form of Function Overloading and Operator Overloading.
9	Examples of POP are: C, VB, FORTRAN, and Pascal.	Example of OOP are: C++, JAVA, VB.NET, C#.NET.



1.6 BASICS OF OOP: ABSTRACTION, INHERITANCE, ENCAPSULATION, CLASSES, SUBCLASSES AND SUPER CLASSES, POLYMORPHISM, MESSAGE COMMUNICATION

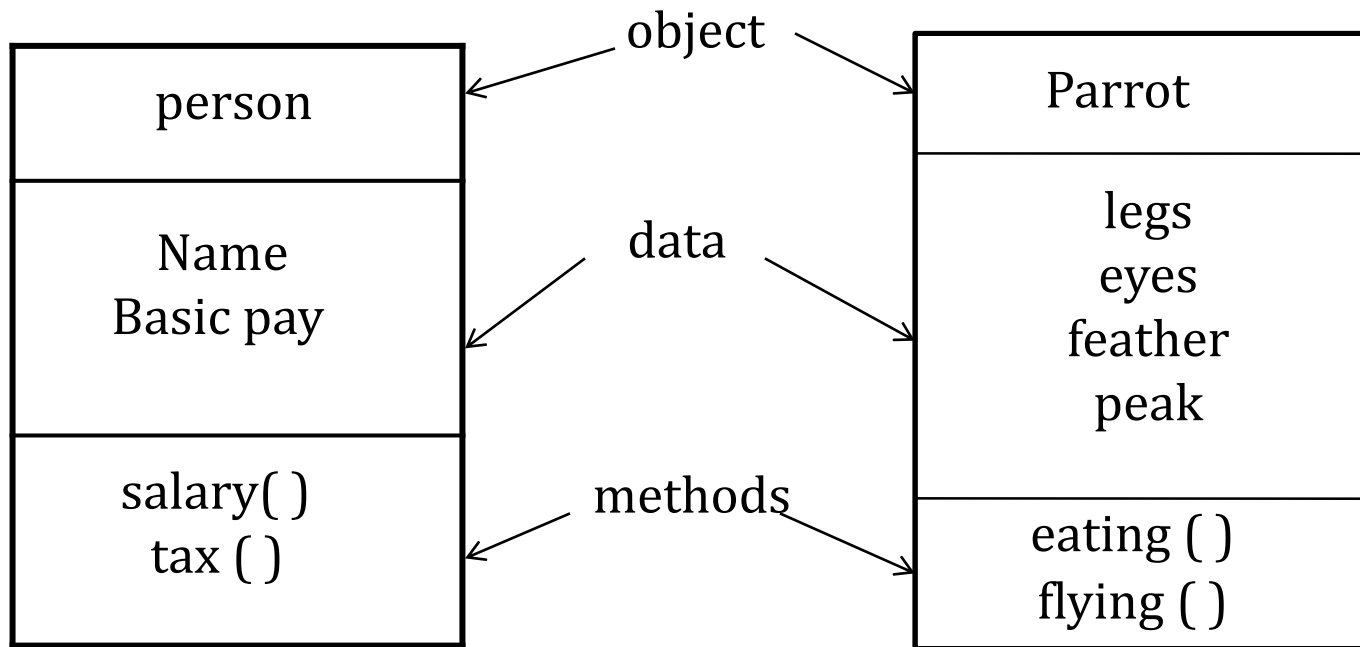


BASIC CONCEPT OF OBJECT ORIENTED PROGRAMMING

1) Object and classes

- ❑ Objects are basic runtime entities in object oriented system.
- ❑ Object may represent a person, place, chair, or any item.
- ❑ Object can interact without having to know the details of each other's data or code.
- ❑ Object is basically a set of data and code to perform operation on data

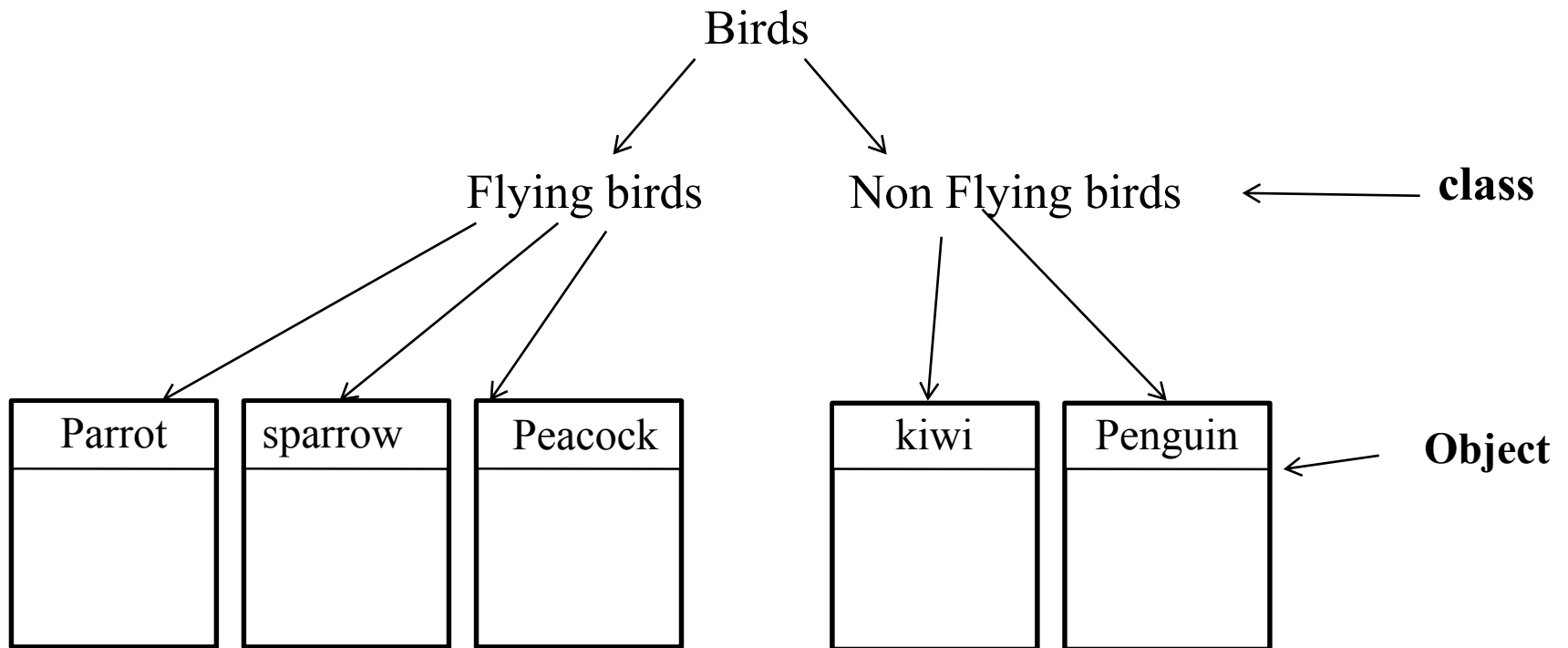




2) CLASS

- ❑ Class is prototype that defines the variables and the methods common to all object of similar type.
- ❑ Classes are user defined data types and behave like built in type of programming language.
- ❑ Simply, Class is a collection of logically related data items which includes data (variable) and function (methods) for data.





3)Data abstraction

and

Encapsulation:

- ❑ **Data abstraction** refers to act of representing essential features without including background details or explanation.

```
abstract class Animal {  
    // Abstract method (does not have a body)  
    public abstract void animalSound();  
}
```

- ❑ **Data encapsulation** means wrapping up of data and methods into a single unit.
- ❑ The data is not accessible to the outside and only these methods, which are wrapped in class access it this insulation of the data from direct access by program is called **data hiding**.

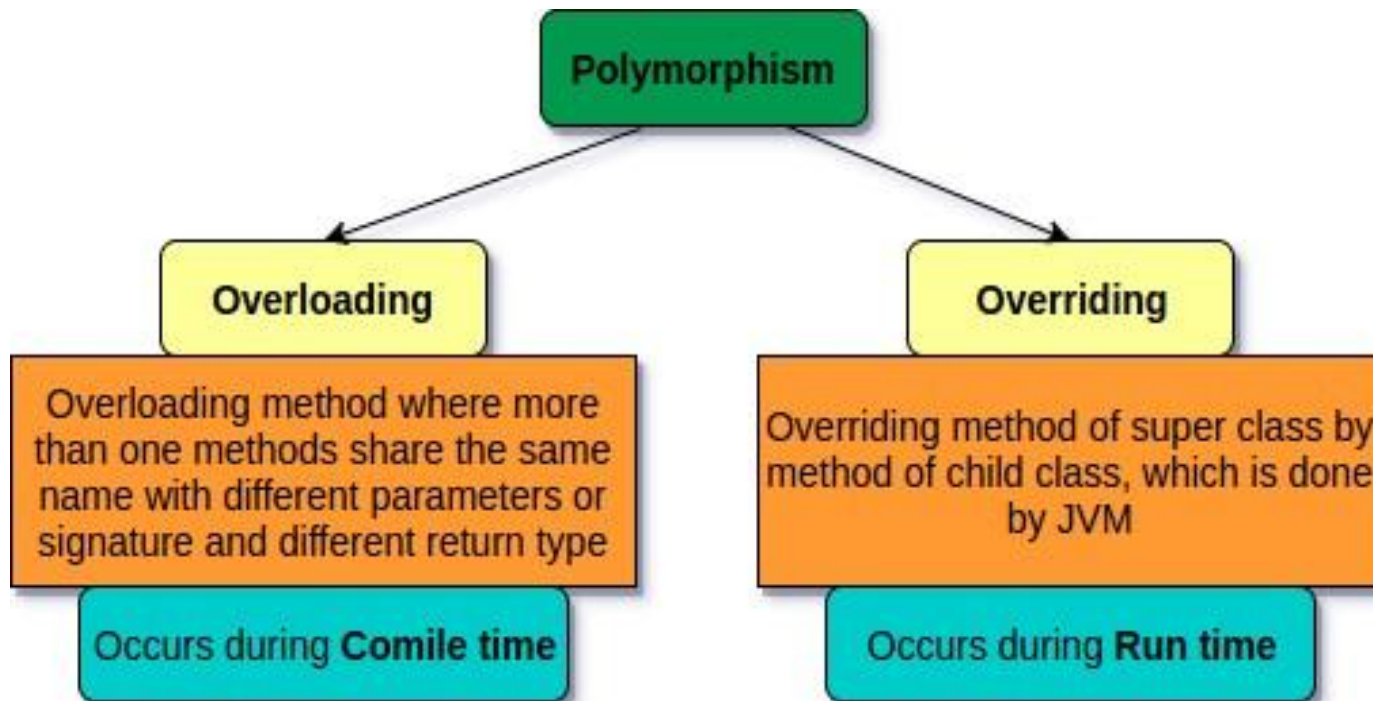
4) Inheritance

- ❑ **Inheritance** is a process by which object of one class use the properties of object of another class.
- ❑ It supports hierarchical classification.
- ❑ It provides Reusability of code. We can add an additional feature to an existing class without modifying it .This is possible by deriving a new class from the existing one.
- ❑ New class with additional features can be created, that new class with have combined features of both the new classes.



5) Polymorphism

Polymorphism means the ability to take more than one form. Simply, it means that are the same operation have may behave different on different classes.



6) Dynamic Binding

- Binding refers to the linking of procedure call to the code to be executed in response to call.
- Dynamic binding means code associated with a given procedure call is not known until the time of call at runtime.
- It is associated with polymorphism and inheritance.

7) Message Communication

- Object oriented program consists of a set of objects that communicate with one another by sending and receiving information.



1.7 COMPILING AND RUNNING A SIMPLE "HELLO WORLD" PROGRAM: SETTING UP YOUR COMPUTER, WRITING A PROGRAM, COMPILING, INTERPRETING AND RUNNING THE PROGRAM, COMMON ERRORS



SIMPLE JAVA PROGRAM

```
class Test
{
    public static void main(String args[ ])
    {
        System.out.println( "welcome to Java");
    }
}
```



1) First line : **class Test**

- declares a class which is an object oriented construct.
- java is true object oriented language and therefore everything must be placed inside a class.
- **class** is keyword and it declares that a new class definition follows. **Test** java identifier(name) specifies name of the class to be defined.

2) Second line : **Opening brace : {**

3) Third Line : **public static void main (String args[]**

Every java program must include main() method. It is the starting point for interpreter to begin the execution of program.



MAIN () METHOD OF JAVA

- **public:** The keyword public is an access modifier. It declares main method as unprotected .it makes method accessible to all other classes.
- **static :** It is a keyword, which declare method is for entire class and not a part of any object of class. The main must always be declared as static since the interpreter uses this method before any objects are created.
- **void** does not return any value.
- **String args** declares parameter names **args** which contain an array of object of class type string.



4) Output line:

System.out.println ("Hello");

class **object** **method**



- This line is similar to `printf ( )` statement of C or `cout<<` construct of C++.
- Since java is true object oriented, every method must be part of an object.
- **println** method is a member of the **out** object, which is a data member of a **System** class. **println** method always append a new line character to the end of the string. This means subsequent output will start on a new line.
- Semicolon at the end of each statement is used. **Every java statement must end with semicolon.**



5) Fifth line :**closing brace :}**

It is the ending of program.

- Note: java program must name same as in class which includes **main() method otherwise interpreter cannot run that class.**



DIFFERENCE BETWEEN PRINT () AND PRINTLN () METHOD

print () method: print and wait

- This method sends information into a buffer. This buffer is not flushed until a newline (or end of line) character sent. As a result **print () method** prints out on one line until a new character is encountered.

`System.out.print ( "hello");`

- We can also use `\n` to force the display to be brought to the next line by printing a newline character as follows:
- `System.out.print ( "hello \n" );` //this would work same as `println ( )` method



println () method : prints a line and move to next line

- This method takes the information provided and displays it on a line followed by a line feed.

```
System.out.println( "java !!!");
```



COMMAND LINE ARGUMENT

- If we want, our program to act in a particular way depending on the input provided at the time of execution.
- This is done in java programs by using command line arguments.
- Definition: **command line arguments are parameters that are supplied to the application program at the time of on invoking it for execution.**
- We can write java program that can receive and use the arguments provided in the command line.

public static void main (String args[])



- **args** is declared as an array of strings(known as string object) .
- Any arguments provided in the command line are passed to the array **args** as its elements.
- We can simply access the array elements and use them in the program as we want.
- Command line arguments are passed when we run the program

To compile : `javac Test.java`

To run : `java Test Basic C++ C Java`

- This command line contains four arguments .These arguments are assigned to the array **args** as follows:

Basic	□	args[0]
C++	□	args[1]
C	□	args[2]
Java	□	args[3]



```
class Sample {  
    public static void main(String args[]) {  
        int count = args.length;  
        System.out.println("Total: " + count);  
  
        int i = 0;  
        while (i < count)  
{  
            System.out.println("Java is " + args[i]);  
            i = i + 1;  
        }  
    }  
}
```



E: \java program \ javac Cmd1.java

E: \java program \ java Cmd1 **simple robust secure
portable dynamic object_oriented**

Output :

total no. Of arguments passed= 6

1 : Java is simple !!!

2 : Java is robust !!!

3 : Java is secure !!!

4 : Java is portable !!!

5 : Java is dynamic !!!

6: Java is object_oriented !!!



READ STATEMENT

- We may also give values to variables interactively through the keyboard using **readLine()** method.
- **readline()** is invoked using an object of the class **DataInputStream**.
- **readLine()** reads the input from the keyboard as a string which is then converted to the corresponding data type using the data wrapper classes.
- We have used the keywords **try** and **catch** to handle any errors that might occur during the reading process.



Example :

```
import java.io .DataInputStream;
class Reading
{ public static void main (String args [ ])
{
DataInputStream obj=new DataInputStream(System.in); int
    i=0;
    float f=0.0 F;
    try
    {
        System.out.println ("enter an integer no "); i =
        Integer.parseInt ( obj. readLine( ) );
        System.out.println ("enter float no ");
        f = Float.valueOf (obj.readLine ( ) ).floatValue( );
    }
}
```



```
catch(Exception e)
{
    System.out.println ( "enter proper value" );
}
System.out.println( "int no i= "+ i );
System.out.println( "float no f= " +f);
}
}
```



OUTPUT:

```
E : \ java program \ javac Reading.java E : \
java program \ java Reading
```

```
enter an integer no
```

```
56
```

```
enter float no
```

```
23.455
```

```
int no i= 56
```

```
float no f=23.455
```



Using Console Class

```
public class Sample {  
    public static void main(String[] args)  
    {  
        String name = System.console().readLine();  
  
        System.out.println("You entered string " + name);  
    }  
}
```

Using Scanner class

```
import java.util.Scanner;

class GetInputFromUser {
    public static void main(String args[])
    {
        // Using Scanner for Getting Input from User
        Scanner in = new Scanner(System.in);

        String s = in.nextLine();
        System.out.println("You entered string " + s);

        int a = in.nextInt();
        System.out.println("You entered integer " + a);

        float b = in.nextFloat();
        System.out.println("You entered float " + b);
    }
}
```

Thank you

